

Neurofive Solutions - Machine Learning Internship Track

Welcome to my machine learning internship repository! This project portfolio demonstrates hands-on implementation of core data science, supervised learning, data cleaning pipelines, ensemble methods, and handling imbalanced datasets using Python and Scikit-Learn.



Table of Contents

1. [Project Overview](#)
 2. [Milestones & Key Projects](#)
 3. [Tech Stack & Libraries](#)
 4. [Project Structure](#)
 5. [Live Web App](#)
-



Project Overview

Throughout this internship track, I progressed from foundational exploratory data analysis and single predictive models to advanced ensemble architectures (Bagging and Boosting) and production-grade techniques for handling real-world imbalanced data (SMOTE).



Milestones & Key Projects

Exploratory Data Analysis & Baseline Models

- **Objective:** Clean datasets, engineer meaningful features, and build initial baseline models (Logistic Regression & Linear Regression).
- **Key Tasks:** Missing value imputation, categorical encoding, feature scaling, and performance benchmarking.

Titanic Survival Prediction - Machine Learning Track

- **Data Cleaning:** Dropped irrelevant or highly sparse columns (`Cabin`, `Name`, `Ticket`, `PassengerId`). Imputed missing `Age` values with the median and `Embarked` with the mode.
- **Encoding:** Converted categorical variables (`Sex`, `Embarked`) into numerical format using Pandas One-Hot Encoding (`pd.get_dummies`).
- **Model Training:** Split data into an 80/20 training/test split and trained a `LogisticRegression` classifier.
- **Evaluation:** Evaluated performance using accuracy metrics and a confusion matrix, achieving ~80-81% accuracy on the test dataset.

Titanic Survival Prediction - Advanced ML Track

- **Evaluation Metrics:** Added `classification_report` tracking Precision, Recall, and F1-score to handle class distribution balance safely.
- **Hyperparameter Tuning:** Implemented `GridSearchCV` to optimize the regularization parameter `C` and solver algorithms, documenting performance variations before and after tuning.

Ensemble Methods (Random Forest vs. XGBoost)

- **Objective:** Compare single-model baselines against powerful ensemble architectures to reduce variance and optimize predictive boundaries.
 - **Key Techniques:**
 - **Random Forest (Bagging):** Built independent trees in parallel and averaged predictions via majority vote.
 - **XGBoost (Boosting):** Sequentially built optimized trees to correct residual errors from previous iterations.
 - **Feature Importance Analysis:** Visualized and compared how each algorithm weighs critical variables (e.g., gender, passenger class, fare ratios).
- | Model | Type | Evaluation Metric | Score |
|---------------------|----------------------------------|-------------------|--------|
| Logistic Regression | Single Model (Linear Classifier) | Accuracy | 79.89% |
| Random Forest | Ensemble (Bagging) | Accuracy | 78.21% |
| XGBoost | Ensemble (Boosting) | Accuracy | 77.65% |
- Interpretation

Logistic Regression achieved the highest accuracy (79.89%) on the Titanic dataset, outperforming both ensemble methods in this experiment. Random Forest achieved 78.21%, while XGBoost achieved 77.65%. This demonstrates that ensemble methods do not always outperform simpler models; their effectiveness depends on the dataset, feature engineering, and hyperparameter tuning. With further tuning (e.g., adjusting tree depth, learning rate, or number of estimators), Random Forest and XGBoost may achieve better performance

Imbalanced Data Handling & Evaluation Metrics

- **Objective:** Address real-world class imbalance (such as fraud detection and customer churn) where traditional accuracy becomes misleading.
 - **Techniques Used:** Applied **SMOTE (Synthetic Minority Over-sampling Technique)** to balance training class distributions.
 - **Evaluation Shift:** Evaluated performance using **Precision, Recall, F1-Score, and Confusion Matrices** rather than relying solely on accuracy.
-



Tech Stack & Libraries

- **Language:** Python
 - **Data Manipulation & Analysis:** Pandas, NumPy
 - **Machine Learning:** Scikit-Learn, XGBoost, Imbalanced-Learn (**imblearn**)
 - **Data Visualization:** Matplotlib, Seaborn
 - **Version Control:** Git & GitHub
-



Repository Structure

```
├─ credit_card_fraud_analysis.py  # Imbalanced data handling & SMOTE pipeline
├─ ensemble_comparison.py        # Random Forest & XGBoost model comparison
├─ housing_regression.py         # Housing price regression models
├─ telco_churn_analysis.py       # Telco customer churn EDA and classification
├─ .gitignore                   # Git exclusion rules (large datasets excluded)
├─ README.md                    # Project documentation
└─ Live Web App
Explore the live interactive machine learning application here:
<a href="https://neurofive-churn-app.streamlit.app" target="_blank">Customer Churn Pr
```